PHP for Beginners — Lesson 1

Nov 8, 2026 · @Jacques MB

By the end of this lesson, learners can write and run a small PHP page that reads a form, makes decisions and prints a result.

Lesson overview

Item	Details
Audience	Complete beginners who know a little HTML
Duration	About 3 hours (or two 90-minute sessions)
PHP version	PHP 8.x
Tools	A code editor (VS Code) and PHP installed locally (XAMPP, Laragon or plain PHP)
Format	Short explanation, live demo, then learners practise

Learning objectives. Learners will be able to:

1. Explain what PHP does and where it runs.
2. Write a PHP file and run it in the browser.
3. Use variables, data types, operators and strings.
4. Control flow with `if`, `match` and loops.
5. Store data in indexed and associative arrays.
6. Write and call their own functions.
7. Read form input safely with `$_POST` and `htmlspecialchars()`.

Suggested timing

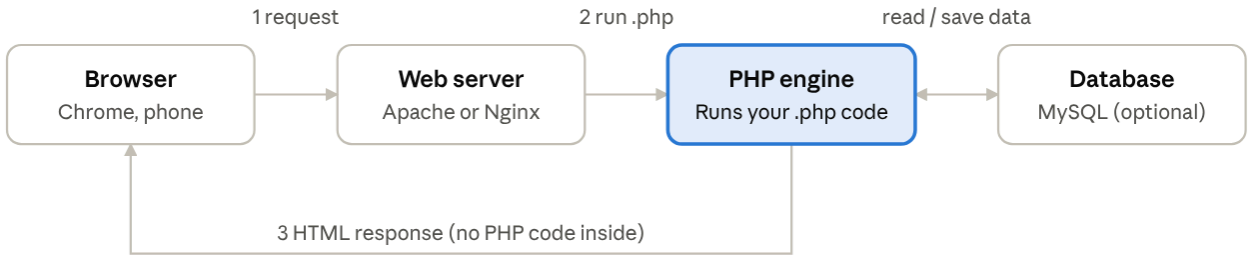
Part	Minutes
What is PHP + setup	25
Syntax, variables, operators, strings	40
Conditionals and loops	35
Break	10

Part	Minutes
Arrays and functions	35
Forms	20
Mini project	25
Quiz and wrap-up	10

1. What is PHP?

PHP is a server-side language: the code runs on the web server and only the resulting HTML reaches the browser. The visitor never sees your PHP code.

PHP runs on the server; the browser only receives HTML



how a PHP page is served · 3 steps

The browser asks for a page, the server hands the .php file to the PHP engine, and only the finished HTML travels back.

- **PHP** stands for "PHP: Hypertext Preprocessor".
- It powers WordPress, Laravel apps, and a large share of websites.
- It mixes easily with HTML, so you can build dynamic pages fast.

Teacher tip: compare it to a restaurant. The browser is the customer, the server is the kitchen, PHP is the cook. The customer gets the finished plate (HTML), never the recipe (PHP).

HTML vs PHP

	HTML	PHP
Runs where	In the browser	On the server
Content	Static, same for everyone	Dynamic, can change per visitor

	HTML	PHP
Can talk to a database	No	Yes
File extension	`.html`	`.php`

2. Setting up

Learners need PHP installed and one folder to work in; the quickest route on Windows is Laragon or XAMPP.

1. Install **Laragon** or **XAMPP** (both include PHP and a web server).
2. Start the web server (Apache) from the control panel.
3. Create a folder `php-lesson` inside `htdocs` (XAMPP) or `www` (Laragon).
4. Create a file `index.php` in that folder.
5. Open `http://localhost/php-lesson/` in the browser.

Alternative (any OS with PHP installed): open a terminal in the folder and run the built-in server:

```
php -v              # check PHP is installed
php -S localhost:8000
```

Then visit `http://localhost:8000`.

Your first PHP file (`index.php`):

```
<!DOCTYPE html>
<html>
<body>
    <h1><?php echo "Hello, World!"; ?></h1>
    <p>Today is <?php echo date("l, d F Y"); ?></p>
</body>
</html>
```

Refresh the page: the date changes every day without editing the file. That is "dynamic". Ask learners to right-click → View Source and notice the PHP code is gone.

3. Syntax basics

Every PHP block starts with `<?php`, every statement ends with a semicolon, and `echo` prints output.

```
<?php
// A single-line comment
# Also a single-line comment
/*
   A multi-line comment
*/

echo "Hello, Kigali!";      // prints text
echo "<br>";                // you can print HTML too
print "Learning PHP";       // print works like echo
```

Rules to remember

- Code goes between `<?php` and `?>`. In a file that is pure PHP, leave off the closing `?>`.
- Each statement ends with `;`. A missing semicolon is the most common beginner error.
- Keywords (`echo`, `if`) are not case-sensitive, but variable names are: `$name` and `$Name` are different.
- Use `<br>` or `\n` (inside double quotes) for new lines.

Common error to show on purpose: remove a semicolon and read the "Parse error: syntax error" message together, including the line number.

4. Variables and data types

A variable is a named box that holds a value; in PHP every variable name starts with `$`.

```
<?php
$name = "Aline";          // string
$age = 21;                // integer
$height = 1.68;           // float
$isStudent = true;        // boolean
$nothing = null;          // null (no value)

echo "Name: $name<br>";
echo "Age: $age<br>";

var_dump($height);        // shows type and value: float(1.68)
echo gettype($isStudent); // boolean
```

Naming rules: start with `$` then a letter or underscore; letters, numbers and `_` only; no spaces. Prefer readable names like `$totalPrice`, not `$tp`.

Type	Example	Used for
string	`"Hello"`	Text
int	`42`	Whole numbers
float	`3.14`	Decimal numbers
bool	`true` / `false`	Yes/no decisions
array	`[1, 2, 3]`	Lists of values
null	`null`	Nothing / empty

Constants never change once set:

```
<?php
const SCHOOL = "Kigali Tech Academy";
define("PASS_MARK", 50);
echo SCHOOL . " pass mark: " . PASS_MARK;
```

Practice: create variables for your name, city, age and favourite food, then print one sentence using all of them.

5. Operators and strings

Operators do maths and comparisons; the dot `.` joins strings together.

Kind	Operators	Example	Result
Arithmetic	`+ - * / % **`	`10 % 3`	`1`
Assignment	`= += -= *= .=`	`$x += 5`	adds 5 to `$x`
Comparison	`== === != !== < > <= >=`	`5 === "5"`	`false`
Logical	`&& || !`	`true && false`	`false`
Increment	`++ --`	`$i++`	adds 1
String join	`.`	`"Hi " . "there"`	`"Hi there"`

Teach this clearly: `==` compares values only, `===` compares value **and** type. Encourage `===` by default.

```
<?php
$price = 1500;
$qty = 3;
$total = $price * $qty;
echo "Total: " . $total . " RWF<br>";   // Total: 4500 RWF

$first = "Jean";
$last = "Mugisha";
$full = $first . " " . $last;
```

Single vs double quotes

```
<?php
$city = "Kigali";
echo "I live in $city";   // I live in Kigali  (variables are read)
echo 'I live in $city';   // I live in $city   (printed as-is)
```

Useful string functions

```
<?php
$text = "learning php is fun";
echo strlen($text);              // 19
echo strtoupper($text);          // LEARNING PHP IS FUN
echo ucwords($text);             // Learning Php Is Fun
echo str_replace("fun", "easy", $text);
echo str_contains($text, "php") ? "yes" : "no"; // yes
```

6. Conditionals

Conditionals let the program choose what to do: `if` / `elseif` / `else` for ranges and `match` for exact values.

```
<?php
$marks = 72;

if ($marks >= 80) {
    echo "Excellent";
} elseif ($marks >= 50) {
    echo "Pass";
} else {
    echo "Fail";
}
```

Combining conditions

```
<?php
$age = 20;
$hasId = true;

if ($age >= 18 && $hasId) {
    echo "You can register.";
}
```

Ternary (short if)

```
<?php
$status = ($marks >= 50) ? "Pass" : "Fail";
```

match (PHP 8) — cleaner than `switch` for exact values:

```
<?php
$day = "Sat";

$type = match ($day) {
    "Sat", "Sun" => "Weekend",
    "Mon", "Tue", "Wed", "Thu", "Fri" => "Weekday",
    default => "Unknown",
};
echo $type; // Weekend
```

Practice: given `$temperature`, print "Cold" below 15, "Warm" from 15 to 25, and "Hot" above 25.

7. Loops

Loops repeat code: use `for` when you know how many times, `while` when you repeat until a condition changes, and `foreach` for arrays.

for — a known number of times:

```
<?php
for ($i = 1; $i <= 5; $i++) {
    echo "Line $i<br>";
}
```

while — until the condition is false:

```php
<?php
$balance = 1000;
$months = 0;
while ($balance < 2000) {
    $balance *= 1.10;   // grows 10% each month
    $months++;
}
echo "Doubled after $months months"; // 8
```

foreach — every item in an array:

```php
<?php
$fruits = ["mango", "banana", "avocado"];
foreach ($fruits as $fruit) {
    echo "I like $fruit<br>";
}
```

break and continue

```php
<?php
for ($i = 1; $i <= 10; $i++) {
    if ($i === 3) continue;  // skip 3
    if ($i === 7) break;     // stop at 7
    echo $i . " ";           // 1 2 4 5 6
}
```

Watch out: a `while` loop whose condition never becomes false runs forever. Always change the variable inside the loop.

Practice: print the multiplication table of 7 (7 × 1 up to 7 × 10).

8. Arrays

An array stores many values in one variable: indexed arrays use numbers as keys, associative arrays use names.

Indexed array (keys start at 0):

```php
<?php
$colors = ["red", "green", "blue"];
echo $colors[0];          // red
$colors[] = "yellow";      // add to the end
echo count($colors);      // 4
```

Associative array (named keys):

```php
<?php
$student = [
    "name" => "Eric",
    "age" => 19,
    "course" => "Web Development",
];
echo $student["name"];   // Eric

foreach ($student as $key => $value) {
    echo "$key: $value<br>";
}
```

Array of arrays (like rows in a table):

```php
<?php
$students = [
    ["name" => "Eric", "marks" => 78],
    ["name" => "Diane", "marks" => 91],
    ["name" => "Patrick", "marks" => 45],
];

foreach ($students as $s) {
    echo $s["name"] . " scored " . $s["marks"] . "<br>";
}
```

Handy array functions

Function	What it does
`count($a)`	Number of items
`in_array("red", $a)`	Is the value in the array?
`sort($a)`	Sort ascending
`array_sum($a)`	Add all numbers
`implode(", ", $a)`	Join into a string
`print_r($a)`	Show the array (for debugging)

Practice: store five numbers in an array, then print their total and average.

9. Functions

A function is a reusable block of code with a name: define it once, call it as many times as you need.

```
<?php
function greet(string $name): string
{
    return "Hello, $name!";
}

echo greet("Aline");   // Hello, Aline!
echo greet("Eric");    // Hello, Eric!
```

Parts of a function

- `function` keyword and a name (`greet`).
- **Parameters** in brackets: the inputs (`string $name`).
- **Return type** after the colon: what comes out (`: string`).
- `return` sends the result back to the caller.

Default values

```
<?php
function calculateTotal(float $price, int $qty = 1): float
{
    return $price * $qty;
}

echo calculateTotal(1500);      // 1500
echo calculateTotal(1500, 4);   // 6000
```

Variable scope: a variable created outside a function is not visible inside it. Pass it in as a parameter instead.

```
<?php
$tax = 0.18;
function addTax(float $amount, float $rate): float
{
    return $amount + ($amount * $rate);
}
echo addTax(10000, $tax);   // 11800
```

Practice: write `isEven(int $n): bool` and use it in a loop to print the even numbers from 1 to 20.

10. Handling forms

PHP reads what users type into HTML forms through `$_GET` (data in the URL) and `$_POST` (data in the request body); always escape it before printing.

	`$_GET`	`$_POST`
Where data travels	In the URL (`?name=Eric`)	Hidden in the request
Good for	Search, filters, links	Logins, sign-ups, anything private
Visible / bookmarkable	Yes	No

Example (`greet.php`) — form and processing in one file:

```
<?php
$message = "";

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim($_POST["name"] ?? "");

    if ($name === "") {
        $message = "Please enter your name.";
    } else {
        $message = "Welcome, " . htmlspecialchars($name) . "!";
    }
}
?>
<!DOCTYPE html>
<html>
<body>
    <form method="post">
        <label>Your name:</label>
        <input type="text" name="name">
        <button type="submit">Send</button>
    </form>

    <p><?= $message ?></p>
</body>
</html>
```

Key ideas

- The `name` attribute of the input becomes the key: `name="name"` → `$_POST["name"]`.
- `?? ""` gives a default when the field is missing (null coalescing operator).
- `trim()` removes spaces at the start and end.

- `<?= ... ?>` is a short way to write `<?php echo ... ?>`.
- **Security:** `htmlspecialchars()` stops users from injecting HTML or JavaScript (XSS). Demo it: type `<b>Hi</b>` with and without it.
- Never trust user input. Check it is present and of the right kind before using it.

11. Mini project: Grade calculator

Learners build one page that takes a student's name and three marks, then shows the average, a letter grade and pass/fail. It uses every topic from this lesson.

Grading scale: A ⩾ 80, B ⩾ 70, C ⩾ 60, D ⩾ 50, F below 50. Pass mark is 50.

Build it in steps (let learners try each step before showing the code):

1. Write the `getGrade()` function and test it with a few numbers.
2. Build the HTML form with a loop over the subjects array.
3. Read and validate the input with `$_POST`.
4. Calculate the average with `array_sum()` and `count()`.
5. Display the report.

Full solution (`grades.php`, tested on PHP 8):

```
<?php
function getGrade(float $average): string
{
    return match (true) {
        $average >= 80 => "A",
        $average >= 70 => "B",
        $average >= 60 => "C",
        $average >= 50 => "D",
        default => "F",
    };
}

$subjects = ["Maths", "English", "Science"];
$result = null;
$errors = [];

if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $name = trim($_POST["name"] ?? "");
    $marks = [];

    if ($name === "") {
        $errors[] = "Name is required.";
    }
```

```
    foreach ($subjects as $subject) {
        $value = $_POST[$subject] ?? "";
        if (!is_numeric($value) || $value < 0 || $value > 100) {
            $errors[] = "$subject must be a number from 0 to 100.";
        } else {
            $marks[$subject] = (float) $value;
        }
    }

    if (empty($errors)) {
        $average = array_sum($marks) / count($marks);
        $result = [
            "name" => htmlspecialchars($name),
            "marks" => $marks,
            "average" => round($average, 1),
            "grade" => getGrade($average),
            "status" => $average >= 50 ? "Pass" : "Fail",
        ];
    }
}
?>
<!DOCTYPE html>
<html>
<head>
    <title>Grade Calculator</title>
</head>
<body>
    <h1>Grade Calculator</h1>

    <?php foreach ($errors as $error): ?>
        <p style="color: red;"><?= $error ?></p>
    <?php endforeach; ?>

    <form method="post">
        <p>Student name: <input type="text" name="name"></p>
        <?php foreach ($subjects as $subject): ?>
            <p><?= $subject ?>: <input type="number" name="<?= $subject ?>"
min="0" max="100"></p>
        <?php endforeach; ?>
        <button type="submit">Calculate</button>
    </form>

    <?php if ($result): ?>
        <h2>Report for <?= $result["name"] ?></h2>
        <ul>
```

```
            <?php foreach ($result["marks"] as $subject => $mark): ?>
                <li><?= $subject ?>: <?= $mark ?></li>
            <?php endforeach; ?>
        </ul>
        <p>Average: <?= $result["average"] ?></p>
        <p>Grade: <strong><?= $result["grade"] ?></strong> (<?=
$result["status"] ?>)</p>
    <?php endif; ?>
</body>
</html>
```

Test it: Diane with 85, 72 and 90 should get an average of 82.3, grade A, Pass. Then try an empty name and a mark of 120 to see the errors.

Extensions for fast learners: add more subjects by editing only the array; style the page with Tailwind CSS; show the highest and lowest mark with `max()` and `min()`.

12. Exercises, quiz and homework

Exercises (in class, 2–5 minutes each)

1. Print your name, age and city in one sentence using variables.
2. Given `$a = 17` and `$b = 5`, print their sum, difference, product, quotient and remainder.
3. Write an `if` that prints "Adult" when `$age` is 18 or more, otherwise "Minor".
4. Use a `for` loop to print the numbers 10 down to 1.
5. Make an associative array for a phone (brand, model, price) and print each key and value.
6. Write a function `square(int $n): int` and print the squares of 1 to 5.

Quick quiz

#	Question	Answer
1	Where does PHP code run?	On the server
2	Which symbol starts every variable name?	`$`
3	Which operator joins two strings?	`.` (dot)
4	What does `5 === "5"` return?	`false` (different types)
5	What is the first index of an indexed array?	`0`

#	Question	Answer
6	Which loop is best for going through an array?	`foreach`
7	Which superglobal holds data from a `method="post"` form?	`$_POST`
8	Which function protects output against XSS?	`htmlspecialchars()`
9	What does `echo 'Hi $name';` print?	`Hi $name` (single quotes do not read variables)
10	What does `strlen("Rwanda")` return?	`6`

Homework: build a "Shopping cart total" page. Store three products (name, price in RWF, quantity) in an array of arrays, loop through them in an HTML table, and show the grand total with 18% VAT added, using a function for the VAT.

Next lesson: include files (`include` , `require`), sessions and cookies, connecting to MySQL with PDO, and an introduction to OOP (classes and objects) — the foundation for frameworks like Laravel.

Free practice resources: php.net manual, W3Schools PHP, and PHP The Right Way.